

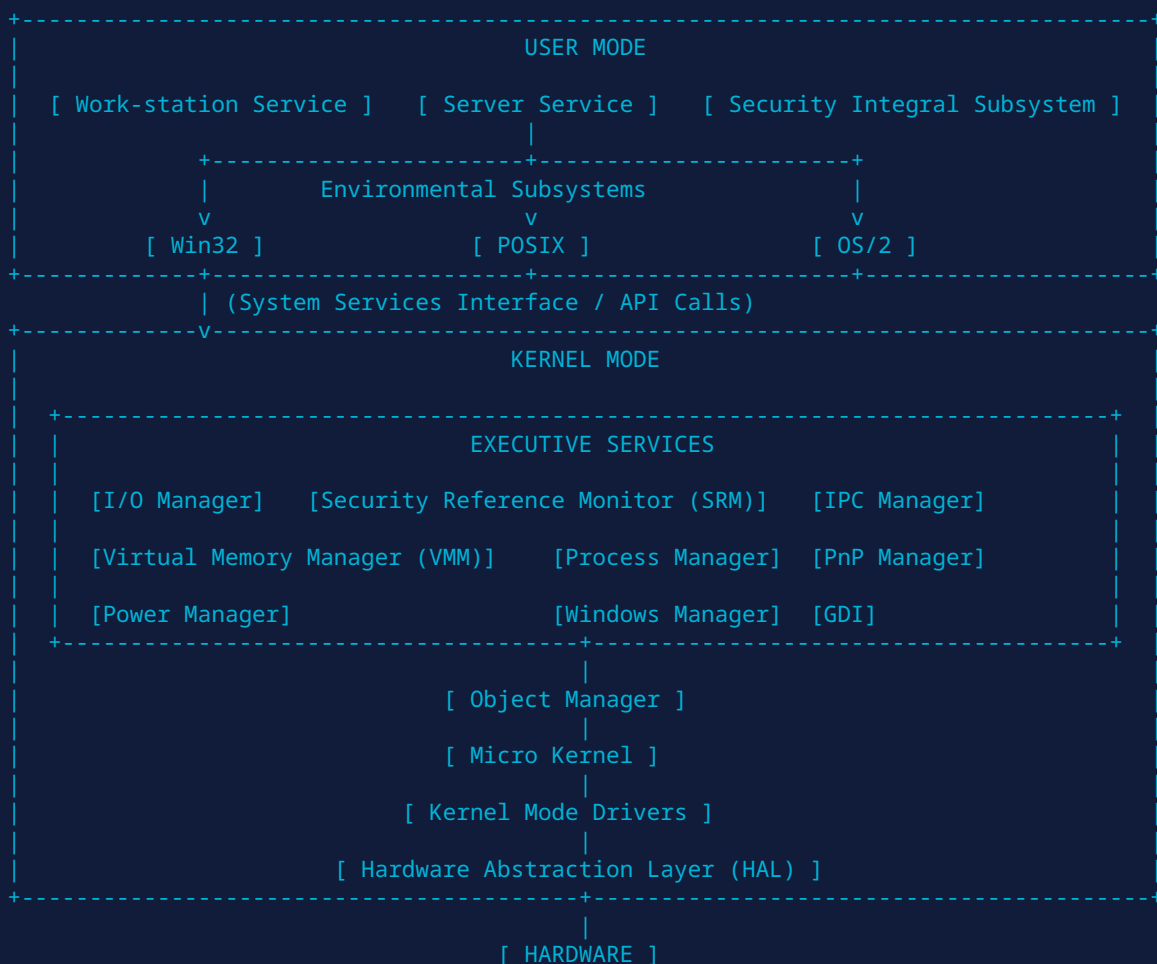
Windows OS Architecture & Registry Systems

1. Windows Architecture Overview

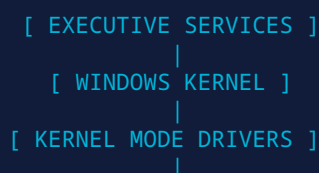
What is Windows Architecture? Windows architecture defines core operating system operational behaviors, specifically how system processes run, how system memory is handled/distributed, how corporate users authenticate credentials, and how overall system security is actively enforced.

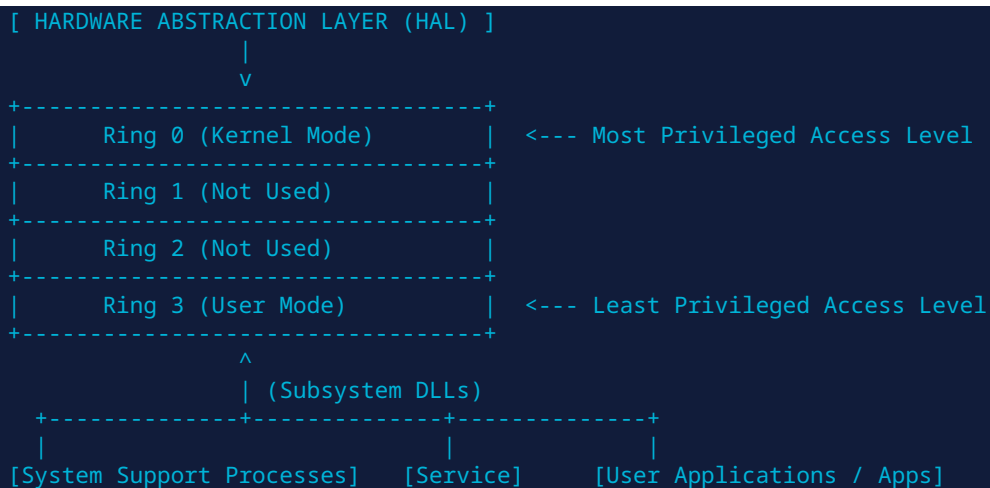
SOC Relevance: In a Security Operations Center, understanding this design identifies exactly where attacks take place.

Windows System Component Layout Diagram



Privilege Rings Mapping Diagram





User Mode vs. Kernel Mode (Two Core Modes)

User Mode (Ring 3 - Least Privileged)

- **Runs:** User applications (e.g., Google Chrome, Microsoft Word), standard user processes, and select software services.
- **Restrictions:** Cannot directly access hardware assets and cannot modify the underlying OS kernel.
- **SOC Relevance:** Most initial malware strains execute here. For example: Phishing Email → Word Macro → PowerShell Execution.

Kernel Mode (Ring 0 - Most Privileged)

- **Runs:** The core Windows Kernel, system-level device drivers, and the hardware memory manager.
- **Power:** Possesses full, unrestricted system access.
- **SOC Relevance:** Advanced malware like Rootkits operate here. A kernel compromise equals complete, unhindered system control.

2. Windows Core Components & Subsystems

Windows Kernel

Handles CPU scheduling, system hardware interrupts, and active thread execution. It is rarely targeted directly by entry-level threats, but a compromise leads to highly stealthy, persistent malware.

Executive Layer Components

- **Process Manager:** Creates and terminates system processes. *SOC Use:* Monitor for abnormal process spawning chains.
- **Memory Manager:** Allocates and manages virtual/physical system memory. *SOC Use:* Advanced fileless malware structures reside and hide here.
- **I/O Manager:** Handles direct disk writes/reads and network communication streams. *SOC Use:* Detect unusual file modifications or suspicious network activity.
- **Security Reference Monitor (SRM):**** The primary security engine of the operating system. It validates object access rights (e.g., "Can this specific user open this file?"). **SOC Use:* Crucial for identifying privilege escalation and unauthorized access attempts.

Hardware Abstraction Layer (HAL) & Drivers

- **HAL:** Serves as the software interface layer between the OS code and the physical hardware devices. It is frequently abused by advanced kernel-level rootkits.
- **Device Drivers:** Software packages that allow the OS to communicate with physical hardware. Threat actors target vulnerable drivers for privilege escalation or engage in Signed Driver Abuse via BYOVD (Bring Your Own Vulnerable Driver).

3. Windows Process & Memory Architecture

Process Architecture

A Windows process is an executing instance of a live program. It acts as an isolated resource container holding virtual memory allocations, object handles, and execution threads needed to run applications. Every process is assigned a unique Process Identifier (PID).

Must-Know System Processes

- **smss.exe (Session Manager Subsystem):** The very first user-mode process that starts up within the system.
- **wininit.exe (Windows Initialization Process):** Initializes critical system background infrastructure.
- **services.exe (Service Control Manager):** Manages, starts, and stops background Windows services.
- **lsass.exe (Local Security Authority Subsystem Service):** Handles user authentications and secure credential storage. *Attack Vector:* Targeted by tools like Mimikatz to dump plaintext passwords and hashes from memory.
- **svchost.exe (Service Host Process):** A generic host process for services running from dynamic-link libraries (DLLs). Malware frequently abuses it or names malicious files similarly to hide in plain sight.

SOC Detection Use-Case

```
[ winword.exe ] (Microsoft Word) ----> Spawns ----> [ powershell.exe ] OR [ cmd.exe ]
```

Analysis: This process chain indicates malicious macro execution originating from a phishing document.

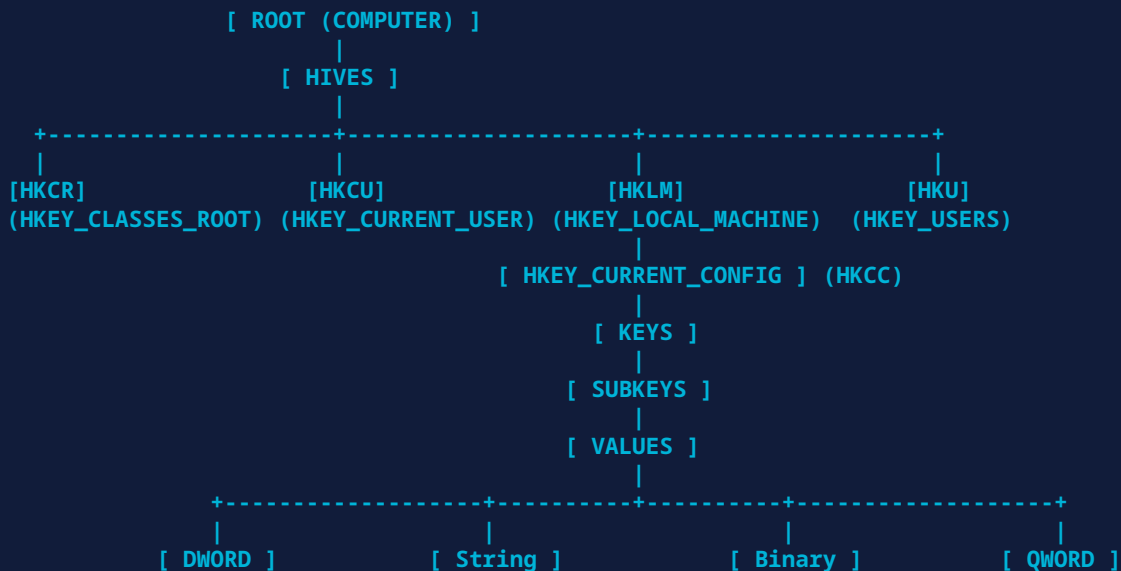
Virtual Memory & Authentication Flow

- **Virtual Memory:** Provides isolated memory spacing to each process to prevent cross-process data tampering. Key SOC threats include process injection, DLL injection, and fileless malware.
- **Access Token:** Contains the user's explicit identity and access privileges. Threat actors attempt token theft or token impersonation to achieve privilege escalation.

4. Windows Registry Architecture

What is the Windows Registry? The Registry is a hierarchical central database used by Windows to store system configurations, core user settings, and application data.

Registry Hierarchical Structure Chart



Structural Path Breakdown

- **Hive:** The top-level root keys of the registry.
- **Key → Subkey:** The organizational folder structures within a hive.
- **Value:** The actual data records stored within keys.

Standard Root Keys (Hives) Reference

- **HKLM (HKEY_LOCAL_MACHINE):** Stores system-wide configuration data for the local computer.
- **HKCU (HKEY_CURRENT_USER):** Stores configuration settings specific to the currently logged-in user.
- **HKCR (HKEY_CLASSES_ROOT):** Stores file association properties and OLE registration data.
- **HKU (HKEY_USERS):** Contains configuration profiles for all user accounts loaded on the system.
- **HKCC (HKEY_CURRENT_CONFIG):** Tracks volatile hardware profile configurations used during boot.

How the Registry Works

1. **System Boot:** Windows reads registry hives → loads system configurations → initializes background services → applies user profiles.
2. **Application Launch:** The application queries specific registry keys → loads software configurations → executes user preferences.

3. Common Value Types: REG_SZ (String data), REG_DWORD (Numeric values), and REG_BINARY (Raw binary data).

5. Registry Exploitation, Attacks & Forensics

Common Registry Attack Tactics

Boot Persistence

Malware adds its executable path into the system auto-start keys to maintain a presence across reboots.

- **Key Locations:** HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- **RunOnce Key:** Executes a command exactly once and then deletes itself; frequently used by loaders and installers.
- **Mechanism:** malware.exe is added as a value, forcing it to execute automatically every time the victim logs into the computer.

Services Modification

- **Location:** HKLM\SYSTEM\CurrentControlSet\Services
- **Mechanism:** Malware installs itself directly as a background Windows service or modifies paths of legitimate services to intercept execution.

Advanced Persistence

Utilizing Windows Management Instrumentation (WMI) event subscriptions to execute malware based on system triggers. Additionally, Registry Hijacking replaces legitimate software or DLL references with malicious file paths.

SOC Forensic Detection Checklist:

When monitoring registry modifications, look for these key anomalies:

- **Suspicious Value Entries:** Look for commands like powershell.exe -nop -w hidden embedded inside standard Run keys.
- **Unknown Executables:** Unrecognized files (e.g., random.exe) configured to auto-start.
- **Unusual File Paths:** Binaries executing from non-standard user spaces like AppData\Local\Temp\ instead of Program Files.
- **Forensic Extraction Capabilities:** Analysts can extract historical records of installed software, recent user activity timelines, and overall execution history directly from registry files.

Summary: The Windows Attack Lifecycle

Real-World SOC Monitoring Alert Example:

- Event ID: 4657 (A registry value was modified)
- Target Key: HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- New Value: malware.exe
- Analysis Result: Active persistence mechanism detected and flagged.

Final SOC Analyst Mindset:

When triaging any security alert on a Windows host, always verify five core questions:

Which process ran? • What files or configurations did it modify? • Is there any registry persistence established? • Was there an attempt at credential access? • Is there any sign of lateral movement?